

typarium

A Typst package for expressive font specimen cards with custom themes and custom renderers.

System fonts, raw local font bytes, Typst 0.15+ paths, metadata dictionaries, and fully custom rendering workflows.

Version 0.1.1

1. Package Surface	1
2. Quick Start Workflow	3
3. <code>font-showcase</code> Parameter Reference	5
4. Accepted <code>fonts</code> Shapes and Normalization Rules	8
5. Resolved Render Payload Contract	13
6. Custom Theme and Renderer Design Guide	16
7. Per-Font Dictionary Keys	21
8. Theme Dictionary Reference	21
9. WASM Metadata Schema (<code>wasm-plugin/src/lib.rs</code>)	24
10. Usage Instructions and Worked Examples	28
11. Behavioral Notes and Caveats	37
12. Internal Helper Summary	38
13. Practical Recommendations	38

1. Package Surface

The typarium package centers on a single high-level function, `#font-showcase(...)`, defined in `lib.typ`. It turns one or more font descriptions into specimen cards, can auto-extract metadata from raw font bytes or Typst 0.15+ path values through `font_parser.wasm`, and can be restyled either by overriding a theme dictionary or by replacing the renderer entirely.

1.1. Public Symbols

The package currently exposes the following practical entry points:

- `lib.typ`
 - `font-showcase`: primary user-facing function.
 - `set-variations`: host helper that attaches normalized variable-axis requests to a font item or theme dictionary.
 - `_parse-font-file`: internal helper that reads a local font file and requests JSON metadata from the WASM plugin.
 - `_resolve-font-meta`: internal helper that normalizes user input, attaches extracted metadata, and prepares the render payload.
- `themes/default.typ`
 - `default-theme`: base design token dictionary used by `font-showcase`.
 - `default-render`: bundled renderer used whenever `render` is not a function.
- `themes/terminal.typ`
 - `terminal-theme`: small override dictionary for a terminal-style presentation.
 - `terminal-render`: bundled alternate renderer with a command-line visual metaphor.

- `themes/anatomy.typ`
 - `anatomy-theme` : override dictionary for technical/anatomy cards with layout tokens.
 - `anatomy-render` : bundled alternate renderer that emphasizes metrics and source information.
- `themes/inspector.typ`
 - `inspector-theme` : bundled token dictionary for glyph and table inspection cards.
 - `inspector-render` : bundled alternate renderer that visualizes `glyph-details`, codepoint samples, capabilities, variation requests, and table diagnostics.

The underscore-prefixed helpers are documented here for completeness, but they should be treated as implementation details rather than stable public API.

1.2. Minimal Import Patterns

For the default experience, pass the user-local font as raw bytes from the calling document. On Typst 0.15.0 or later, you can pass a resolved `path(...)` value instead and let `typarium` read it inside the package.

```
#import "@preview/typarium:0.1.1": font-showcase
```

```
#font-showcase(
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none)),)
)
```

Typst 0.15.0+ path-based equivalent:

```
#font-showcase(
  fonts: ((path: path("Jaldi-Regular.ttf")),)
)
```

Visual result:

Jaldi
Regular 400
STATIC

Designed by Pablo Cosgaya and Nicolas Silva

Minimal import, immediate specimen output.

About this font

Jaldi is the devanagari version of Asap wich is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standarised character width on all styles which helps retain the same length on all text lines.

For alternate bundled themes:

```
#import "@preview/typarium:0.1.1": font-showcase, terminal-theme, terminal-render
```

```
#font-showcase(
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none))),
  theme: terminal-theme,
  render: terminal-render,
)
```

Visual result:

```
root@typst-os:~# fetch-font-meta --name "Jaldi"
> LOADING METADATA... OK. > AUTHOR: Pablo Cosgaya and Nicolas Silva > TYPE: Static >
VERSION: Version 1.007 > STYLE: 400 / normal
```

```
root@typst-os:~# render-sample --target "Jaldi"
```

BOOT

Whereas recognition of the inherent dignity

```
root@typst-os:~# _
```

1.3. Real Function Signature

The exact signature in `lib.typ` is:

```
#let font-showcase(
  fonts: auto,
  theme: (:),
  render: auto,
  columns: 1,
) = context { ... }
```

`font-showcase` is intentionally primitive. Its job is to normalize font inputs, parse font bytes from raw bytes or Typst 0.15+ path sources through the WASM plugin, prepare a `font-text` helper, merge showcase-level and per-font `theme` overrides, and hand the resolved payload to a renderer.

The bundled `default`, `terminal`, `anatomy`, and `inspector` renderers are sample implementations built on top of this host API. Renderer-facing specimen keys such as `sample-text`, `waterfall`, or `show-details` are therefore carried through `theme` or per-font dictionaries rather than through separate top-level function parameters.

2. Quick Start Workflow

2.1. Choose an Input Shape

You can call `font-showcase` in five common ways:

- Omit `fonts` entirely and let the function use the current Typst text font.
- Pass a string such as `"Helvetica"` to target a system-installed font.
- Pass raw bytes from `read("FontFile.ttf", encoding: none)` either directly in `fonts` or through a font dictionary `path` field. This is the compatibility path for Typst 0.14.x.
- On Typst 0.15.0 or later, pass `path("FontFile.ttf")` either directly in `fonts` or through a font dictionary `path` field. The path is resolved at the call site, then read by `typarium` inside the package.
- Pass a dictionary or an array of dictionaries to attach labels, overrides, metadata, or layout settings per font item. For actual font-file parsing, prefer raw bytes or Typst 0.15+ path values over package-relative string paths.

2.2. Decide Where Metadata Should Come From

The library merges information from four places:

1. Built-in internal defaults required for payload stability.
2. Top-level `theme`.
3. Metadata extracted by `font_parser.wasm` when raw font bytes or a Typst 0.15+ path source are present.

4. Explicit per-font dictionary overrides inside `fonts` .

Later stages override earlier ones, with one exception: the library performs special handling for `name` and `render-name` , documented in Section 4.

2.3. Use a Theme or a Renderer

- If `render` is not a function, the package falls back to `default-render` .
- When `render` is omitted, `theme` is merged with `default-theme` .
- When `render` is a custom function, `theme` is passed through as-is and item-level `theme` dictionaries are layered on top.
- A custom renderer receives a normalized dictionary containing the merged theme, a ready-to-use text function, the merged metadata, and every relevant specimen option.

2.4. Compile-Time Requirements

When a font item provides raw bytes or a Typst 0.15+ path source, `font-showcase` does all of the following:

- accepts raw bytes passed through `path: read("FontFile.ttf", encoding: none)` ;
- accepts Typst 0.15+ path values passed through `path: path("FontFile.ttf")` or directly as `fonts: path("FontFile.ttf")` ;
- reads path sources as raw bytes inside the package;
- sends the bytes into `font_parser.wasm` using `plugin("font_parser.wasm")` ;
- parses the returned JSON into a Typst dictionary.

Typst does not let installed packages read arbitrary project files through package-relative string paths. For Typst 0.14.x compatibility, pass user-local fonts as bytes from the calling document. On Typst 0.15.0 or later, pass a `path(...)` value; callers do not need to wrap that path value in `read(...)` .

2.5. Inspection Limits for `name` -Based Fonts

This distinction is crucial: `typarium` has two very different operating modes.

1. `name` / family-string mode
2. `parsed-file` mode via `read("FontFile.ttf", encoding: none)` bytes or Typst 0.15+ `path("FontFile.ttf")`

In `name` mode, Typst only gives the package a renderable family descriptor for the `text` function. That is enough to typeset specimen text, but it does not reveal which concrete font file Typst selected, where that file lives, or what its binary tables contain. As a result, `typarium` cannot run `font_parser.wasm` on that font and cannot recover file-level metadata from the family name alone.

In `parsed-file` mode, the calling document explicitly supplies the font binary or a resolved path value. That gives `typarium` something concrete to parse, so the package can extract the full WASM metadata surface and forward it into bundled or custom renderers.

Practical consequences of `name` / family-string mode:

- specimen rendering works normally;
- `font-text` still uses the requested family, weight, style, OpenType features, `lang` , and `script` ;
- Typst itself still performs its usual family matching and fallback behavior;
- but parsed metadata is unavailable because no font bytes were provided.

When neither raw bytes nor a Typst 0.15+ path source are available, you should assume all WASM-derived data is absent unless you add it manually. This includes, but is not limited to:

- `glyphs`
- `glyph-details`
- `glyph-name-index`

- `codepoint-samples`
- `number-of-glyphs`
- `styles-count`
- `metrics` extracted from the font binary
- name-table records such as `postscript-name`, `version`, `designer-url`, `vendor-url`, `license-url`, and related fields
- permissions and embedding flags
- table diagnostics
- variation axes and collection-face summaries

This means bundled renderer behavior changes depending on input shape:

- `default-render` can still show title, specimen, paragraph text, and any manual metadata you provide;
- `show-glyphs: true` only renders a glyph section when `glyphs` actually exists;
- detail-oriented renderers such as `anatomy` and `inspector` become much more informative when raw bytes or a Typst 0.15+ path source are supplied;
- a custom renderer should always tolerate missing parsed metadata.

Recommended interpretation:

- use `name:` or a plain family string when you only need rendering;
- use raw bytes or Typst 0.15+ path values when you need inspection, glyph analysis, technical metadata, or reproducible parser output;
- if you intentionally stay in `name` mode, provide any important metadata manually through the per-font dictionary.

This is not a limitation of `typarium` alone. It follows from Typst’s package and font model together with the fact that the `text` function accepts family descriptors for shaping, not an inspectable font-file handle.

3. `font-showcase` Parameter Reference

3.1. Core Parameters

Name	Type	Default	Behavior
<code>fonts</code>	<code>auto</code> <code>str</code> <code>bytes</code> <code>path</code> <code>dictionary</code> <code>array</code>	<code>auto</code>	Controls which fonts become specimen cards. <code>auto</code> uses the current <code>text.font</code> context. A single string, bytes value, Typst 0.15+ path value, or dictionary is automatically wrapped into a one-item array. Use raw bytes for Typst 0.14.x compatibility, or <code>path(...)</code> on Typst 0.15.0+ for user-local fonts.
<code>theme</code>	<code>dictionary</code>	<code>(:)</code>	Showcase-level renderer configuration and design-token payload forwarded into the active renderer layer. When <code>render</code> is omitted, it is merged into <code>default-theme</code> ; when <code>render</code> is custom, it is passed through and then combined with per-font <code>theme</code> overrides.
<code>render</code>	<code>auto</code> <code>function</code>	<code>auto</code>	If this is a function, it is called once for every resolved font item. Otherwise the package uses <code>default-render</code> .

<code>columns</code>	<code>int</code>	<code>1</code>	Forwarded to Typst <code>grid(columns: ...)</code> as <code>(1fr,) * columns</code> . The package does not validate the number, so use a positive integer.
----------------------	------------------	----------------	--

3.2. Common Bundled Renderer Keys

Name	Type	Default	Behavior
<code>sample-text</code>	<code>content str</code>	"Whereas recognition of the inherent dignity"	Primary specimen text used by bundled renderers unless overridden per item. Supply it through <code>theme</code> or per-font dictionaries.
<code>sample-size</code>	<code>length</code>	<code>2.0em</code>	Used by <code>default-render</code> when no <code>waterfall</code> sizes are supplied.
<code>hero-text</code>	<code>none content str</code>	<code>none</code>	Optional oversized display text for bundled renderers.
<code>sub-text</code>	<code>none content str</code>	<code>none</code>	Optional secondary specimen content.
<code>paragraph-text</code>	<code>none content str</code>	<code>none</code>	Optional paragraph specimen block for bundled renderers.
<code>description</code>	<code>none content str</code>	<code>none</code>	Optional editorial description block.
<code>align</code>	<code>alignment</code>	<code>left</code>	Alignment used by bundled specimen blocks. Typical values are <code>left</code> , <code>center</code> , and <code>right</code> .
<code>sample-fallback</code>	<code>bool</code>	<code>true</code>	Passed into the prepared <code>font-text</code> function as Typst <code>fallback</code> . Set this to <code>false</code> when you want missing glyphs to remain missing instead of falling back to another font.
<code>waterfall</code>	<code>array</code>	<code>()</code>	A sequence of sizes such as <code>(1.2em, 1.8em, 2.4em)</code> . When non-empty, the default renderer draws one sample row per size and ignores <code>sample-size</code> for the main specimen block.
<code>show-glyphs</code>	<code>bool</code>	<code>false</code>	Enables the glyph section in the default renderer when metadata contains <code>glyphs</code> . Family-name inputs alone do not produce <code>glyphs</code> ; for that, you need raw font bytes, a Typst 0.15+ path source, or manual metadata.
<code>leading</code>	<code>length</code>	<code>0.8em</code>	Controls line spacing for the main specimen block, waterfall rows, and glyph section. Reduce this when narrow cards cause wrapped specimen lines to feel too loose.
<code>show-details</code>	<code>bool dictionary</code>	<code>false</code>	When enabled, the default renderer adds a metadata detail section and bundled alternate renderers honor the related visibility flags. A

			dictionary can override individual detail rows such as <code>author</code> , <code>manufacturer</code> , <code>license</code> , <code>font-type</code> , <code>weight</code> , <code>width</code> , <code>styles-count</code> , <code>number-of-glyphs</code> , <code>postscript-name</code> , <code>version</code> , <code>copyright</code> , and <code>license-url</code> .
<code>title-overflow</code>	<code>auto</code> <code>"inline"</code> <code>"stack"</code>	<code>auto</code>	Controls how the default renderer handles long titles. <code>auto</code> measures the title block and badge block, then stacks metadata only when the inline header would overflow in narrow multi-column cards.
<code>title-overflow-badge-position</code>	<code>"top"</code> <code>"bottom"</code>	<code>"top"</code>	When a stacked header is used, controls whether badges appear above the title block or below the title and author block.
<code>show-author</code>	<code>bool</code>	<code>true</code>	Shows or hides author metadata in bundled renderers.
<code>show-badges</code>	<code>bool</code>	<code>true</code>	Shows or hides style and type badges in the default renderer.
<code>show-description</code>	<code>bool</code>	<code>true</code>	Shows or hides the “About this font” panel when description content exists.
<code>show-postscript-name</code>	<code>bool</code>	<code>true</code>	Controls PostScript-name visibility in detail-oriented bundled renderers.
<code>show-version</code>	<code>bool</code>	<code>true</code>	Controls version visibility in bundled renderers.
<code>show-copyright</code>	<code>bool</code>	<code>true</code>	Controls copyright visibility in bundled renderers.
<code>show-license-url</code>	<code>bool</code>	<code>true</code>	Controls license URL visibility in bundled renderers.
<code>features</code>	<code>dictionary</code>	<code>(:)</code>	OpenType feature settings forwarded into the prepared <code>font-text</code> helper.
<code>lang</code>	<code>auto</code> <code>str</code>	<code>auto</code>	Language shaping control forwarded into <code>font-text</code> .
<code>script</code>	<code>auto</code> <code>str</code>	<code>auto</code>	Script shaping control forwarded into <code>font-text</code> .
<code>variation-values</code>	<code>dictionary</code>	<code>(:)</code>	Optional variable-axis requests resolved by the host into <code>variation-request</code> . These are exposed to renderers now, but Typst 0.14.x still does not apply them to <code>font-text</code> natively.

3.3. Important Precedence Rules

There are two different precedence layers in the package.

Host-layer metadata resolution happens inside `font-showcase` itself:

1. Built-in host defaults such as `column-count`.
2. Extracted metadata from the font file, when a `path` is present.

3. Explicit per-font dictionary keys inside `fonts`.

Renderer-layer option resolution is then expected to happen like this:

1. Per-font overrides from `it.item-overrides`.
2. Showcase-level `theme`.
3. Host-resolved metadata already present on `it`.
4. Literal fallback inside the renderer.

Practical examples:

- `theme: (sample-text: ...)` should beat an embedded naming-table `sample_text` unless the per-font item overrides it again.
- A font file can still supply structural metadata such as `postscript-name`, `version`, `variations`, or `permissions` when neither the item nor the theme provides those keys.
- A per-font `show-glyphs: true` can enable glyph rendering for only one item in a multi-font grid because it lives in `item-overrides`.

4. Accepted `fonts` Shapes and Normalization Rules

4.1. `set-variations(target, values)`

`set-variations` is the host-side helper for variable-axis requests. It does not make Typst apply variable font axes yet; instead it normalizes the request into `variation-values`, and `font-showcase` turns that into a structured `variation-request` payload for renderers. The warning you may see for variable fonts comes from Typst itself rather than from this package. Using a static instance of the same family can remove that warning, but it still does not give Typst true variable-axis shaping.

Typical usage:

```
#let item = set-variations(
  (
    name: "Skia",
    font-type: "Variable",
    variations: (
      axes: (
        (tag: "wght", min-value: 0.48, default-value: 1.0, max-value: 3.2, hidden: false),
        (tag: "wdth", min-value: 0.62, default-value: 1.0, max-value: 1.3, hidden: false),
      ),
      coordinates: (0, 0),
      has-non-default-coordinates: false,
    ),
  ),
  (wght: 1.6, wdth: 1.1),
)
```

The resulting `it.variation-request` contains:

- `values` : normalized axis request dictionary
- `matched-axes` : parsed axis records augmented with `requested-value` and `requested-normalized`
- `unmatched-tags` : tags requested by the user but missing from the parsed axis list
- `native-support` : currently `false`
- `applied-in-font-text` : currently `false`
- `mode` and `note` : explanatory host-level status strings

That makes it possible to build meaningful variable-font UI now, even before Typst itself can apply axis values in shaping.

4.2. fonts: auto

When `fonts` is `auto`, the library reads the surrounding `text.font` context and uses that as the target font input. This is the most lightweight way to preview the active document font.

```
#set text(font: "Libertinus Serif")
#font-showcase()
```

libertinus serif

Regular 400

Whereas recognition of the inherent dignity

4.3. Plain Font Family String

A plain string that does not look like a font file path is treated as a renderable font family name.

```
#font-showcase(fonts: "Helvetica")
```

Normalization result:

- `render-name` becomes the string itself.
- `name` becomes the same string.
- no WASM metadata is extracted.
- `weight/style` fall back to `regular` and `normal` unless later overridden.

Visual result:

Helvetica

Regular 400

System family string input.

No local file parsing is involved here.

4.4. Raw Font Bytes

A bytes value from `read("FontFile.ttf", encoding: none)` is the compatibility-safe way to parse a user-local font file from an installed package, including on Typst 0.14.x.

```
#font-showcase(
  fonts: read("Jaldi-Regular.ttf", encoding: none),
  theme: (sample-fallback: false),
)
```

Normalization result:

- the bytes are parsed by the WASM plugin;
- `render-name` defaults to the extracted family name;
- `name` defaults to the same extracted family name;
- `theme` still provides fallback values for renderer-oriented keys such as `sample-text` or `paragraph-text`.

Visual result:

Jaldi

Designed by Pablo Cosgaya and Nicolas Silva

Regular 400

STATIC

Raw bytes input triggers WASM metadata extraction.

About this font

Jaldi is the devanagari version of Asap wich is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standarised character width on all styles which helps retain the same length on all text lines.

4.5. Typst 0.15+ Path Value

A `path("FontFile.ttf")` value is resolved relative to the file where it is constructed. This lets a project pass a resolved project-local font path to typarium, and typarium can read and parse that path inside the package.

```
#font-showcase(  
  fonts: path("Jaldi-Regular.ttf"),  
  theme: (sample-fallback: false),  
)
```

Normalization result:

- the path is read by typarium, then parsed by the WASM plugin;
- `render-name` defaults to the extracted family name;
- `name` defaults to the same extracted family name;
- callers pass `path(...)` directly; they do not need to use `read(path(...))`.

Visual result:

Jaldi

Designed by Pablo Cosgaya and Nicolas Silva

Regular 400

STATIC

Typst path input triggers package-side reading and parsing.

About this font

Jaldi is the devanagari version of Asap wich is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standarised character width on all styles which helps retain the same length on all text lines.

4.6. Dictionary Without `path`

A dictionary without a `path` key is interpreted as a manually described font item.

```
#font-showcase(  
  fonts: (  
    name: "Libertinus Serif",  
    display-name: "Libertinus Serif (Editorial Label)",  
    author: "Philipp H. Poll",  
    sample-text: "Manual metadata without file parsing.",
```

```
)  
)
```

Special handling for this case:

- `render-name` is assigned from `name` when present, otherwise from the current context font.
- `name` is assigned from `display-name` when present, otherwise from `render-name`.
- this split is useful when the actual font family and the card label should differ.

Visual result:

Libertinus Serif

Designed by Philipp H. Poll

Regular 400

Manual metadata without file parsing.

The visible card title can differ from the render family.

4.7. Dictionary With `path`

A dictionary with `path` triggers file parsing, then applies the rest of the dictionary as explicit overrides. The `path` value may be raw bytes from `read(..., encoding: none)`, or on Typst 0.15.0+, a resolved path from `path(...)`.

```
#font-showcase(  
  fonts: (  
    path: read("Jaldi-Regular.ttf", encoding: none),  
    name: "Jaldi (Custom Card Title)",  
    sample-text: "File metadata plus explicit overrides.",  
    show-glyphs: true,  
  )  
)
```

Special handling for this case:

- `render-name` initially comes from the parsed font metadata;
- `name` defaults to the parsed `render-name`, unless the dictionary sets `name`;
- all non-`path` keys are merged in after parsing;
- if you need to override the actual rendered family separately from the visible card label, provide both `render-name` and `name` explicitly.

On Typst 0.15.0 or later, this is equivalent:

```
#font-showcase(  
  fonts: (  
    path: path("Jaldi-Regular.ttf"),  
    name: "Jaldi (Custom Card Title)",  
    sample-text: "File metadata plus explicit overrides.",  
    show-glyphs: true,  
  )  
)
```

Visual result:

Jaldi (Custom Card Title)

Designed by Pablo Cosgaya and Nicolas Silva

Regular 400

STATIC

File metadata plus explicit overrides.

Numbers

0 1 2 3 4 5 6 7 8 9

Latin Uppercase

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Latin Lowercase

a b c d e f g h i j k l m n o p q r s t u v w x y z

About this font

Jaldi is the devanagari version of Asap wich is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standarised character width on all styles which helps retain the same length on all text lines.

4.8. Array of Mixed Items

Arrays may contain strings, raw bytes, Typst 0.15+ path values, dictionaries, or a mixture of these. Each entry becomes one card.

```
#font-showcase(  
  columns: 2,  
  fonts: (  
    "Libertinus Serif",  
    (path: read("Jaldi-Regular.ttf", encoding: none), sample-text: "Parsed from file"),  
    (name: "Courier", display-name: "Courier (Manual Label)"),  
  ),  
)
```

The package wraps a single string, bytes value, Typst 0.15+ path value, or dictionary into an array automatically, so you only need explicit parentheses when you truly want multiple items.

Visual result:

Libertinus Serif Regular 400

Whereas recognition
of the inherent dignity

Jaldi Regular 400 STATIC

Designed by Pablo Cosgaya and Nicolas Silva

Parsed from file

About this font

Jaldi is the devanagari version of Asap which is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standardised character width on all styles which helps retain the same length on all text lines.

Courier Regular 400

Whereas recog-
nition of the
inherent dig-
nity

5. Resolved Render Payload Contract

Every card renderer receives one dictionary, referred to here as `it`. The library constructs it as:

```
let font-text = text.with(  
  font: meta.render-name,  
  weight: meta.weight,  
  style: meta.style,  
  fallback: meta.sample-fallback,  
)  
let card-theme = base-theme + meta.at("theme", default: ({}))  
let it = meta + (font-text: font-text, theme: card-theme)
```

5.1. Guaranteed Keys

The following keys are always present after normalization, even if they only carry built-in or theme-derived values:

- `theme`: merged theme dictionary.
- `font-text`: prepared helper function for typesetting specimen text in the target font.
- `render-name`: actual font family (or font family array) used by `font-text`.
- `name`: human-facing card label.
- `weight`: defaults to `"regular"`, but may become a numeric class from parsed metadata.
- `style`: defaults to `"normal"`.
- `hero-text`, `sample-text`, `sub-text`, `paragraph-text`, `description`.

- `sample-size`, `sample-fallback`, `waterfall`, `show-glyphs`, `show-details`.
- `leading`: defaults to `0.8em` and can be overridden globally or per item.
- `align`: resolved alignment key used by bundled specimen blocks.
- `features`, `lang`, `script`.
- `title-overflow`, `title-overflow-badge-position`, `show-author`, `show-badges`, `show-description`.
- `show-postscript-name`, `show-version`, `show-copyright`, `show-license-url`.

5.2. Optional Metadata Keys From WASM Parsing

When raw font bytes or a Typst 0.15+ path source are parsed successfully, additional keys may appear on `it`. In `name-only` mode, these keys should generally be assumed absent unless you supplied them manually:

- `postscript-name`
- `font-type`
- `width`
- `author`
- `designer-url`
- `manufacturer`
- `vendor-url`
- `version`
- `trademark`
- `license`
- `copyright`
- `license-url`
- `styles-count`
- `number-of-glyphs`
- `metrics`
- `glyphs`

Top-level metadata keys with underscores in the Rust serializer are normalized to hyphenated Typst keys when inserted into `it`.

5.3. Nested Structures

`metrics` is normalized recursively, so nested keys are now hyphenated in the same style as the top-level metadata:

- `units-per-em`
- `ascender`
- `descender`
- `line-gap`
- `x-height`
- `cap-height`
- `bbox`
- `italic-angle`

`glyphs` is an array of dictionaries, each with:

- `name`: category label.
- `chars`: a space-separated string of extracted characters.

5.4. Custom Renderer Example

```
#font-showcase(  
  columns: 2,  
  fonts: (  
    (path: read("Jaldi-Regular.ttf", encoding: none), sub-text: "Parsed locally"),  
    (name: "Courier", display-name: "Courier (System)"),  
  ),  
  render: it => {  
    let ft = it.font-text  
    let overrides = it.at("item-overrides", default: (:))  
    let sample-text = if type(overrides) == dictionary and "sample-text" in overrides {  
      overrides.at("sample-text")  
    } else {  
      it.theme.at("sample-text", default: it.at("sample-text", default: "Whereas recognition  
of the inherent dignity"))  
    }  
    let sample-size = if type(overrides) == dictionary and "sample-size" in overrides {  
      overrides.at("sample-size")  
    } else {  
      it.theme.at("sample-size", default: it.at("sample-size", default: 2.0em))  
    }  
    block(  
      breakable: false,  
      width: 100%,  
      fill: rgb("1f2330"),  
      inset: 1.4em,  
      radius: 0.7em,  
      stroke: 0.5pt + rgb("47506a"),  
      [  
        #text(fill: rgb("dfe7ff"), weight: "bold", size: 1.4em)[#it.name]  
        #v(0.35em)  
        #text(fill: rgb("91a1c6"), size: 0.8em)[  
          family=#it.render-name, style=#it.style, weight=#it.weight  
        ]  
        #v(1em)  
        #ft(size: sample-size, fill: rgb("ffffff"))[#sample-text]  
        #if it.at("glyphs", default: none) != none [  
          #v(1em)  
          #text(fill: rgb("91a1c6"), size: 0.75em)[Glyph categories: #it.glyphs.len()]  
        ]  
      ],  
    ),  
  },  
)
```

Because `render` returns arbitrary content, custom renderers should set `breakable: false` themselves when a whole card should stay on one page.

Visual result:

Jaldi

family=Jaldi, style=normal, weight=400

Courier

family=Courier, style=normal, weight=regular

Whereas recog-
nition of the
inherent dig-
nity

6. Custom Theme and Renderer Design Guide

If you are building on typarium as a host rather than as a ready-made specimen layout, the cleanest mental model is:

- `font-showcase` resolves font inputs, metadata, and shaping helpers.
- `theme` is your renderer's configuration surface.
- per-font dictionaries are your per-card escape hatch.
- `render` is the actual product surface.

In other words, the package is intentionally closer to a render host than to a monolithic preset API.

6.1. Glyph Inspector Renderer

The expanded parser payload now makes it practical to build technical inspectors directly in Typst. This sample renderer uses `glyph-details`, `glyph-name-index`, `codepoint-samples`, and `capabilities` without reparsing the font.

```
#let glyph-inspector-theme = (  
  color-card: rgb("f5f7fb"),  
  color-stroke: rgb("cfd7e6"),  
  color-title: rgb("1f2a3a"),  
  color-muted: rgb("607089"),  
  color-rule: rgb("d9e0ec"),  
  size-title: 1.15em,  
  size-sample: 1.6em,  
  probe-glyph-ids: (0, 1, 2, 3, 4, 5, 6, 7),  
  sample-text: "Glyph metrics inspector",  
)  
  
#let glyph-inspector-render = it => {  
  let ft = it.font-text  
  let overrides = it.at("item-overrides", default: ({}))  
  let opt = (key, default) => if type(overrides) == dictionary and key in overrides {  
    overrides.at(key)  
  } else {  
    it.theme.at(key, default: it.at(key, default: default))  
  }  
  let glyph-details = it.at("glyph-details", default: ({}))  
  let codepoint-samples = it.at("codepoint-samples", default: ({}))
```

```

let glyph-name-index = it.at("glyph-name-index", default: ({}))
let capabilities = it.at("capabilities", default: ({}))
let probe-glyph-ids = opt("probe-glyph-ids", (0, 1, 2, 3))

let cells = (
  [*ID*], [*Name*], [*Advance*], [*Side Bearing*], [*BBox*],
)
for gid in probe-glyph-ids {
  let glyph = glyph-details.at(gid, default: none)
  if glyph != none {
    let bbox = glyph.at("bounding-box", default: none)
    let bbox-label = if bbox == none {
      "_"
    } else {
      str(bbox.at("width", default: "?")) + " x " + str(bbox.at("height", default: "?"))
    }
    cells.push([#gid])
    cells.push([#glyph.at("name", default: "(unnamed)")]
    cells.push([#glyph.at("horizontal-advance", default: "-")]
    cells.push([#glyph.at("horizontal-side-bearing", default: "-")]
    cells.push([#bbox-label])
  }
}

block(
  breakable: false,
  width: 100%,
  fill: opt("color-card", white),
  stroke: 0.5pt + opt("color-stroke", luma(210)),
  inset: 1.1em,
  radius: 0.6em,
  [
    #text(size: opt("size-title", 1.1em), weight: "bold", fill: opt("color-title", black))
[#it.name]
    #v(0.35em)
    #text(size: 0.8em, fill: opt("color-muted", gray))[
      glyphs=#glyph-details.len() / named=#glyph-name-index.len() / codepoint
samples=#codepoint-samples.len()
    ]
    #v(0.9em)
    #ft(size: opt("size-sample", 1.5em), fill: opt("color-title", black))[#opt("sample-
text", "Glyph inspector")]
    #v(0.9em)
    #grid(columns: (auto, auto, auto), gutter: 0.9em,
      [color=#capabilities.at("has-color-glyphs", default: false)],
      [svg=#capabilities.at("has-svg-images", default: false)],
      [raster=#capabilities.at("has-raster-images", default: false)],
    )
    #v(0.9em)
    #grid(
      columns: (auto, 1fr, auto, auto, auto),
      column-gutter: 0.8em,
      row-gutter: 0.35em,
      ..cells,
    )
  ],
)

```

```

}

#font-showcase(
  theme: glyph-inspector-theme,
  render: glyph-inspector-render,
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none)),),
)

```

Jaldi

glyphs=1222 / named=1222 / codepoint samples=256

Glyph metrics inspector

color= **false** svg= **false** raster= **false**

ID	Name	Advance	Side Bearing	BBox
0	.notdef	914	13	888 x 1244
1	NULL	0	0	-
2	nonmarkingreturn	427	0	-
3	space	427	0	-
4	A	1090	22	1046 x 1252
5	Aacute	1090	22	1046 x 1619
6	Abreve	1090	22	1046 x 1588
7	uni1EAE	1090	22	1046 x 1760

6.2. Recommended Design Rules

1. Keep host concerns and renderer concerns separate. Host concerns are things like file parsing, metadata normalization, `font-text`, and grid layout. Renderer concerns are visual hierarchy, optional sections, badge logic, and copy decisions.
2. Treat `theme` as the shared renderer contract. If a knob should affect every card in a showcase, it belongs in `theme`.
3. Treat per-font dictionaries as exceptions, not as the main API. Use per-font keys for card-specific overrides such as one custom sample sentence, one alternate badge state, or one special inset.
4. Resolve options in the order `item -> theme -> metadata -> literal fallback`. This gives card-level overrides without sacrificing showcase-level consistency.
5. Prefer namespaced token families. Keys such as `color-*`, `size-*`, `gap-*`, `stroke-*`, `panel-*`, or renderer-specific prefixes like `poster-*` are easier to grow than flat one-off names.
6. Always render specimen text through `it.font-text`. That preserves the resolved family, weight, style, `features`, `lang`, `script`, and fallback behavior.
7. Decide page-break behavior explicitly. If a card must stay intact, put `breakable: false` on the outermost block returned by the renderer.

6.3. Recommended Option Resolution Pattern

The bundled renderers now use a small helper in this style:

```

#let opt(it, key, default) = {
  let overrides = it.at("item-overrides", default: ())
  if type(overrides) == dictionary and key in overrides {
    overrides.at(key)
  } else {
    it.theme.at(key, default: it.at(key, default: default))
  }
}

```

```
}
}
```

That pattern gives you three useful behaviors at once:

- per-font dictionaries win when a key is present on the resolved item;
- showcase-level `theme` supplies shared renderer defaults;
- parsed metadata is still available when neither of the two renderer-facing layers supplies a value;
- the renderer still has a hard-coded fallback for missing keys.

For custom renderers, this is the single most useful pattern to copy.

6.4. Suggested Token Layout

When designing a custom theme, a practical structure is:

- `color-*` for semantic paint values `color-primary`, `color-muted`, `color-panel`, `color-accent`
- `size-*` for type scales `size-title`, `size-meta`, `size-sample`, `size-caption`
- `gap-*` for vertical and horizontal rhythm `gap-header`, `gap-meta`, `gap-section`, `gap-sample`
- `stroke-*` for rules and borders `stroke-card`, `stroke-divider`
- renderer-specific layout tokens `poster-ratio`, `specimen-panel-width`, `metric-line-gutter`

This keeps the renderer flexible without forcing typarium itself to grow new top-level parameters.

6.5. Worked Custom Renderer

The following pattern is a good starting point for a completely custom renderer:

```
#let poster-theme = (
  color-bg: rgb("0f1720"),
  color-primary: rgb("f8fafc"),
  color-muted: rgb("94a3b8"),
  color-accent: rgb("f59e0b"),
  stroke-card: 0.6pt + rgb("334155"),
  size-title: 1.3em,
  size-meta: 0.78em,
  size-sample: 2.1em,
  gap-meta: 0.4em,
  gap-sample: 0.9em,
  card-inset: 1.2em,
  sample-text: "Custom renderers should read from theme first.",
)
```

```
#let poster-render = it => {
  let ft = it.font-text
  let opt = (key, default) => {
    let overrides = it.at("item-overrides", default: ({}))
    if type(overrides) == dictionary and key in overrides {
      overrides.at(key)
    } else {
      it.theme.at(key, default: it.at(key, default: default))
    }
  }
  let sample-text = opt("sample-text", "Fallback specimen")
  let sample-size = opt("size-sample", 2em)

  block(
    breakable: false,
    width: 100%,
    fill: opt("color-bg", black),
    stroke: opt("stroke-card", none),
    inset: opt("card-inset", 1em),
```

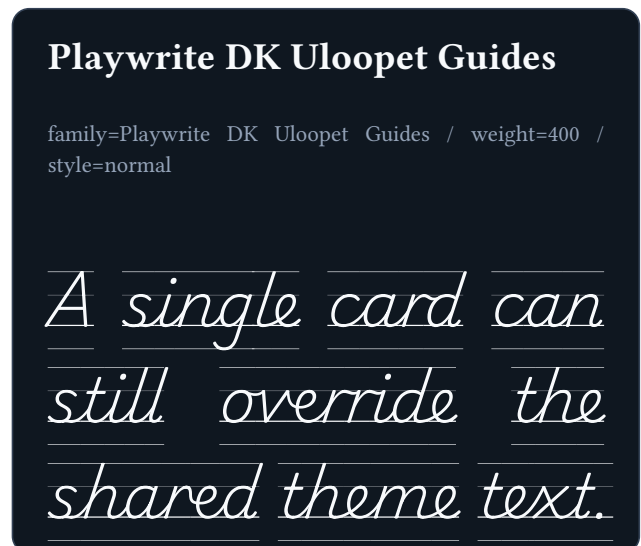
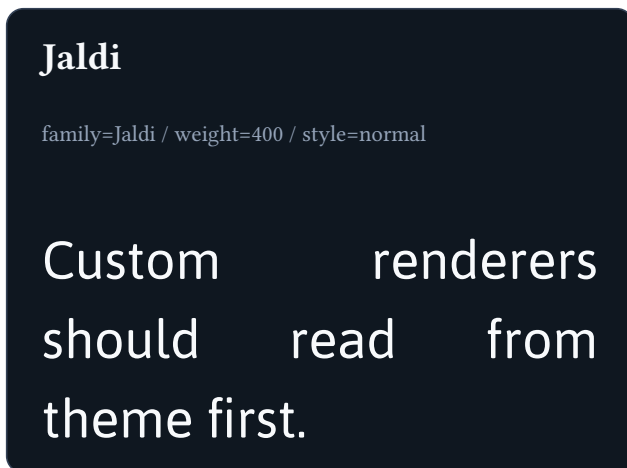
```

radius: 0.6em,
[
  #text(
    size: opt("size-title", 1.2em),
    weight: "bold",
    fill: opt("color-primary", white),
  )[#it.name]
  #v(opt("gap-meta", 0.4em))
  #text(size: opt("size-meta", 0.8em), fill: opt("color-muted", white))[
    family=#it.render-name / weight=#it.weight / style=#it.style
  ]
  #v(opt("gap-sample", 0.8em))
  #ft(size: sample-size, fill: opt("color-primary", white))[#sample-text]
],
)
}

#font-showcase(
  columns: 2,
  theme: poster-theme,
  render: poster-render,
  fonts: (
    (path: read("Jaldi-Regular.ttf", encoding: none)),
    (
      path: read("PlaywriteDKUloopetGuides-Regular.ttf", encoding: none),
      sample-text: "A single card can still override the shared theme text.",
      theme: (color-accent: rgb("22c55e")),
    ),
  ),
)

```

Visual result:



6.6. Practical Architecture Advice

- Put long-lived visual defaults in `theme`, not in the host function.
- Put truly font-specific content in the font item itself.
- If your renderer grows more than about a dozen tokens, document them as a mini theme contract just like `default-theme` and `anatomy-theme` do.
- If multiple renderers share helper logic, keep that helper outside the renderer body and treat it as part of your renderer module's internal API.

7. Per-Font Dictionary Keys

In addition to the top-level function parameters, per-font dictionaries can carry manual metadata and renderer-specific settings. The library does not enforce a strict schema; it simply merges keys into the resolved item dictionary, with `path` receiving special treatment.

Commonly useful per-font keys include:

- `path` : raw font bytes passed from the calling document, or a Typst 0.15+ `path(...)` value resolved in the project.
- `name` : visible label for the card, or the render family for manual entries without `path`. When you use `name` without raw bytes or a Typst 0.15+ `path` source, the card is render-only unless you also provide metadata manually.
- `display-name` : card label for manual entries without `path`.
- `render-name` : explicit render family override.
- `author`, `description`, `license`, `version`, `copyright`.
- `sample-text`, `hero-text`, `sub-text`, `paragraph-text`.
- `weight`, `style`, `sample-fallback`, `show-glyphs`, `waterfall`.
- `features`, `lang`, `script` : typography controls forwarded into bundled and custom renderers.
- `variation-request`, `variation-support` : host-level variable-font request metadata for renderers and inspectors.
- `title-overflow`, `title-overflow-badge-position`, `show-author`, `show-badges`, `show-description`.
- `show-postscript-name`, `show-version`, `show-copyright`, `show-license-url`.
- any additional project-specific keys that your custom `render` callback understands.

7.1. Typography Controls

Bundled renderers now honor:

- `features` for OpenType feature dictionaries
- `lang` for language-sensitive shaping
- `script` for script-sensitive shaping

This means manual dictionary items and `theme` can both drive typography-sensitive regression cases without requiring a custom renderer. However, typography controls do not magically make file-level metadata available; they only affect shaping.

8. Theme Dictionary Reference

The call-time theme is always calculated as `default-theme + theme`, meaning the keys listed below form the full baseline contract for bundled renderers.

8.1. Typography and Color Tokens

Key	Default	Meaning
<code>ui-font</code>	<code>"libertinus serif"</code>	UI font used for labels and metadata rather than specimen text.
<code>color-primary</code>	<code>rgb("202124")</code>	Primary text and headline color for bundled renderers.
<code>color-secondary</code>	<code>rgb("5f6368")</code>	Muted metadata and supporting label color.
<code>color-paragraph</code>	<code>auto</code>	Paragraph color. When <code>auto</code> , the default renderer lightens <code>color-primary</code> by 15 percent.

color-divider	rgb("f1f3f4")	Paint used by the default section divider when stroke-divider is auto.
color-waterfall-divider	rgb("e0e0e0")	Paint used by waterfall row separators when stroke-waterfall is auto.
color-desc-bg	rgb("f8f9fa")	Background color for the description panel in the default renderer.
color-link	auto	Link color. When auto, the default renderer uses a lightened blue.

8.2. Size Tokens

Key	Default	Meaning
size-name	1.6em	Font size of the card title in the default renderer.
size-author	0.9em	Size of the author credit line.
size-badge-label	0.9em	Size of the style/weight badge label.
size-badge-type	0.7em	Size of the boxed font-type badge.
size-hero	6.4em	Size of hero-text.
size-waterfall-label	0.75em	Label size for each waterfall row.
size-sub	0.9em	Size of sub-text.
size-paragraph	0.95em	Paragraph specimen size.
size-glyph-label	0.8em	Category title size in the glyph section.
size-glyph	1.2em	Glyph display size.
size-desc-title	0.9em	Title size inside the description block.
size-desc	0.9em	Description body size.
size-details-copyright	0.8em	Currently defined in the base theme but unused by bundled renderers.
size-details	0.7em	Currently defined in the base theme but unused by bundled renderers.

8.3. Grid and Gap Tokens

Key	Default	Meaning
grid-column-gutter	1.6em	Horizontal spacing between specimen cards.
grid-row-gutter	0em	Vertical spacing between specimen cards.
gap-author	0.8em	Space before the author line.
gap-badges	0.8em	Gap between metadata badges.
gap-header	1.6em	Space below the title/badge header block.
gap-hero	3.2em	Space below the hero specimen.
gap-waterfall-top	0.8em	Extra space inserted before waterfall rows.

gap-waterfall-label	0.8em	Space between a waterfall label and its specimen line.
gap-waterfall-item	1.6em	Gap between successive waterfall rows.
gap-sub	1.2em	Space before the sub-text section.
gap-section	1.6em	Space before paragraph, glyph, or description sections.
gap-section-inner	1.2em	Internal gap after a section divider.
gap-glyph-label	0.6em	Gap after each glyph category title.
gap-glyph-item	0.8em	Gap between glyph categories.
gap-desc-title	0.6em	Gap between the description heading and body.
gap-details	1.2em	Currently defined but unused by bundled renderers.
gap-details-inner	0.8em	Currently defined but unused by bundled renderers.
gap-details-copy	0.6em	Currently defined but unused by bundled renderers.
gap-details-version	0.4em	Currently defined but unused by bundled renderers.
gap-details-license	0.2em	Currently defined but unused by bundled renderers.

8.4. Card, Badge, and Stroke Tokens

Key	Default	Meaning
card-inset	1.6em	Inner padding for each default-render card.
card-radius	0.6em	Card corner radius.
card-stroke	0.5pt + rgb("dadce0")	Border stroke for cards and some badge outlines.
card-spacing	1.6em	Vertical space below each card.
badge-inset	(x: 0.6em, y: 0.3em)	Padding for boxed type badges.
badge-radius	1.0em	Corner radius for boxed type badges.
waterfall-inset	(bottom: 0.6em)	Inset applied to the line above each waterfall sample.
waterfall-tracking	0.05em	Tracking used in waterfall labels.
stroke-divider	auto	Section divider stroke. auto resolves to 0.5pt + color-divider.
stroke-waterfall	auto	Waterfall separator stroke. auto resolves to (bottom: 0.5pt + color-waterfall-divider).
stroke-details	auto	Defined in the base theme but not used by default-render.
desc-inset	1.2em	Inset for the description panel.
desc-radius	0.6em	Corner radius for the description panel.
leading-text	1.4em	Paragraph leading and glyph section leading.

8.5. Alternate Theme Dictionaries

`terminal-theme` is intentionally small and only overrides:

- `ui-font`
- `color-primary`
- `color-secondary`
- `card-radius`
- `card-stroke`
- `card-inset`

`anatomy-theme` now exposes its own layout-oriented tuning surface as well. In addition to color and card tokens such as:

- `ui-font`
- `mono-font`
- `color-primary`
- `color-secondary`
- `color-accent`
- `color-metric`
- `color-bg`
- `color-divider`
- `color-panel-bg`
- `card-stroke`
- `card-radius`
- `card-inset`

it also accepts anatomy-specific layout families such as:

- `size-*`
- `gap-*`
- `metric-*`
- `visual-*`
- `specimen-*`

Those keys matter only when the paired alternate renderers use them.

`inspector-theme` adds inspection-oriented tokens such as:

- `probe-glyph-ids`
- `sample-codepoint-count`
- `mono-font`
- capability badge colors and inspector panel rhythm

9. WASM Metadata Schema (`wasm-plugin/src/lib.rs`)

The Rust parser serializes the following top-level structure before Typst normalizes underscore-containing top-level keys into hyphenated names.

9.1. Top-Level Metadata Fields

Serialized field	Type	Meaning
<code>name</code>	<code>String</code>	Preferred display family name. The parser prefers typographic family names over legacy family names.
<code>family</code>	<code>Option<String></code>	Legacy family name from the naming table.

typographic_family	Option<String>	Typographic family name, if present.
subfamily	Option<String>	Legacy subfamily name.
typographic_subfamily	Option<String>	Typographic subfamily name.
full_name	Option<String>	Full face name.
unique_id	Option<String>	Unique identifier from name-table ID 3 .
postscript_name	Option<String>	PostScript name. Becomes postscript-name in Typst.
postscript_name_prefix	Option<String>	Variation PostScript prefix when supplied by the font.
compatible_full_name	Option<String>	Compatible full name from the naming table.
wws_family	Option<String>	WWS family name.
wws_subfamily	Option<String>	WWS subfamily name.
light_background_palette_name	Option<String>	Suggested palette label for light backgrounds.
dark_background_palette_name	Option<String>	Suggested palette label for dark backgrounds.
font_type	String	"Variable" or "Static". Becomes font-type in Typst.
style	String	"italic", "oblique", or "normal" .
weight	u16	OpenType numeric weight class from ttf-parser .
width	u16	OpenType width class number.
author	Option<String>	Designer name from the font naming table.
designer_url	Option<String>	Designer URL. Becomes designer-url .
manufacturer	Option<String>	Manufacturer string.
vendor_url	Option<String>	Vendor URL. Becomes vendor-url .
version	Option<String>	Version string.
description	Option<String>	Font description.
sample_text	Option<String>	Embedded sample text from the naming table when present.
trademark	Option<String>	Trademark string.
license	Option<String>	License body text.
copyright	Option<String>	Copyright notice from naming-table ID 0 .
license_url	Option<String>	License URL. Becomes license-url .
permissions	PermissionsMeta	Embedding and subsetting permissions extracted from the OS/2 table.
styles_count	u32	Number of fonts in a collection; becomes styles-count .
collection_faces	Option<Vec<CollectionFaceMeta>>	UnionFaceMeta for every face in a TTC/OTC collection.

<code>number_of_glyphs</code>	<code>u16</code>	Glyph count; becomes <code>number-of-glyphs</code> .
<code>metrics</code>	<code>FontMetrics</code>	Expanded metric structure including typographic, vertical, and decoration metrics.
<code>capabilities</code>	<code>CapabilitiesMeta</code>	Boolean capability flags such as variable/color/raster/SVG support.
<code>variations</code>	<code>VariationMeta</code>	Variable-font axes plus the current normalized coordinates.
<code>tables_present</code>	<code>Vec<String></code>	Sorted list of supported OpenType table tags present in the parsed face.
<code>table_sizes</code>	<code>Dictionary<String, usize></code>	Map from table tag to raw byte length for each exposed table.
<code>name_table</code>	<code>NameTableMeta</code>	All readable naming-table records plus a convenience <code>selected</code> map.
<code>glyphs</code>	<code>Option<Vec<GlyphCategory>></code>	Category-level glyph preview lists used by the default renderer.
<code>codepoint_samples</code>	<code>Vec<CodepointGlyphMap></code>	Representative codepoint-to-glyph mappings gathered across common Unicode blocks.
<code>glyph_name_index</code>	<code>Dictionary<String, u16></code>	Map from glyph name to glyph ID for every glyph whose name is known.
<code>glyph_details</code>	<code>Vec<GlyphMeta></code>	Per-glyph metrics and image capability records for every glyph in the face.

9.2. metrics, permissions, capabilities, and variations

`metrics` contains:

- `units_per_em`, `ascender`, `descender`, `height`, `line_gap`
- `typographic_ascender`, `typographic_descender`, `typographic_line_gap`
- `vertical_ascender`, `vertical_descender`, `vertical_height`, `vertical_line_gap`
- `x_height`, `cap_height`, `italic_angle`
- `bbox` as a dictionary with `x_min`, `y_min`, `x_max`, `y_max`, `width`, and `height`
- `underline`, `strikeout`, `subscript`, and `superscript` metric dictionaries when the source tables exist

`permissions` contains:

- `level`: one of `installable`, `restricted`, `preview-and-print`, or `editable` when the value is valid
- `is_subsetting_allowed`
- `is_bitmap_embedding_allowed`

`capabilities` contains:

- `is_regular`, `is_italic`, `is_oblique`, `is_bold`, `is_monospaced`, `is_variable`
- `has_raster_images`, `has_svg_images`, `has_color_glyphs`
- `color_palettes`: palette count for COLR/CPAL fonts when present

`variations` contains:

- `axes`: array of dictionaries with `tag`, `name`, `name_id`, `min_value`, `default_value`, `max_value`, and `hidden`
- `coordinates`: current normalized coordinates from `ttf-parser`, serialized as f2.14 `i16` values
- `has_non_default_coordinates`

9.3. `name_table`, `collection_faces`, and table diagnostics

`name_table.records` keeps every readable naming-table record with:

- `platform_id`, `encoding_id`, `language_id`, `language`
- `name_id`, `name_id_label`
- `is_unicode`, `value`, `raw_length`

`name_table.selected` is a convenience dictionary containing the first preferred decoded value for each name label.

`collection_faces` appears only for TTC/OTC collections and exposes per-face summaries with:

- `index`
- `name`
- `postscript_name`
- `style`, `weight`, `width`
- `number_of_glyphs`
- `is_variable`

`tables_present` and `table_sizes` make it easy for a custom renderer to detect whether features such as COLR, SVG, fvar, MVAR, vhea, or sbix exist before trying to explain them visually.

9.4. `glyphs`, `codepoint_samples`, `glyph_name_index`, and `glyph_details`

`glyphs` is the lightweight preview list used by the bundled default renderer. Each entry has:

- `name`: category label
- `chars`: a space-separated string of extracted characters

`codepoint_samples` gives representative mappings for common Unicode ranges. Each entry contains:

- `char`
- `codepoint`
- `glyph_id`
- `glyph_name`
- `variation_selector`
- `variation_glyph_id`

`glyph_name_index` lets a custom renderer look up a glyph ID from a glyph name without reparsing the font.

`glyph_details` is the exhaustive per-glyph record. Each entry contains:

- `id`
- `name`
- `horizontal_advance`, `vertical_advance`
- `horizontal_side_bearing`, `vertical_side_bearing`
- `y_origin`
- `bounding_box` with the same rectangle structure used in `metrics.bbox`
- `raster_image` with `pixels_per_em`, `format`, `offsets`, `dimensions`, and `data_length` when the glyph exposes bitmap data
- `svg_image` with glyph range, data length, and compression flag when the glyph uses SVG data
- `is_color_glyph`

Because `glyph_details` covers every glyph, it can be large for substantial CJK or emoji fonts. That is intentional: the goal is to give custom renderers enough data to build technical inspectors without needing another parsing pass.

9.5. Glyph Extraction Categories

The lightweight `glyphs` preview list scans the following Unicode blocks and collects up to 100 present characters per block:

- Numbers (U+0030 to U+0039)
- Latin Uppercase (U+0041 to U+005A)
- Latin Lowercase (U+0061 to U+007A)
- Cyrillic (U+0400 to U+04FF)
- Hiragana (U+3040 to U+309F)
- Katakana (U+30A0 to U+30FF)
- Han Ideograms (U+4E00 to U+9FFF)

10. Usage Instructions and Worked Examples

10.1. Basic Local Font Specimen

```
#font-showcase(  
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none)),),  
  theme: (sample-fallback: false),  
)
```

Jaldi

Regular 400 STATIC

Designed by Pablo Cosgaya and Nicolas Silva

Whereas recognition of the inherent dignity

About this font

Jaldi is the devanagari version of Asap wich is based on Ancha (designed by Pablo Cosgaya & Hector Gatti). This family specially developed for screen and desktop use offers of a standarised character width on all styles which helps retain the same length on all text lines.

10.2. Manual System Font Card

```
#font-showcase(  
  fonts: (  
    name: "Libertinus Serif",  
    display-name: "Libertinus Serif (System Font)",  
    author: "Manual metadata",  
    sample-text: "No file parsing is required for system fonts.",  
    sub-text: "Useful for environments where only installed fonts are available.",  
  )  
)
```

Libertinus Serif

Regular 400

Designed by Manual metadata

No file parsing is required for system fonts.

Useful for environments where only installed fonts are available.

10.3. Rich Editorial Layout With Description and Paragraph

```
#font-showcase(  
  fonts: (  
    path: read("PlaywriteDKUloopetGuides-Regular.ttf", encoding: none),  
    hero-text: "AaGg",  
    sample-text: "Learn to write with guided rhythm and generous spacing.",  
    paragraph-text: lorem(30),  
    description: "This card demonstrates the default renderer's hero, paragraph, and  
description regions.",  
  )  
)
```

Playwrite DK Uloopet Guides

Regular 400

STATIC

Designed by Veronika Burian, José Scaglione

AaGg

Learn to write with guided rhythm and generous spacing.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaeerat.

About this font

This card demonstrates the default renderer's hero, paragraph, and description regions.

10.4. OpenType Features and Metadata Toggles

```
#font-showcase(  
  theme: (  
    show-details: (  
      author: true,  
      license: true,  
      font-type: true,  
      weight: true,  
      styles-count: true,  
      postscript-name: true,  
      version: true,  
    ),  
    show-license-url: false,  
  ),  
  fonts: (  
    name: "Libertinus Serif",
```

```

display-name: "Libertinus Serif (Small Caps + Details)",
author: "Philipp H. Poll",
features: ("smcp": 1),
lang: "en",
script: "latn",
sample-text: "OpenType features now affect bundled renderers.",
license: "OFL",
font-type: "otf",
weight: 400,
styles-count: 6,
version: "Manual v1.0",
postscript-name: "LibertinusSerif-Regular",
copyright: "Example copyright line for the details section.",
)
)

```

Libertinus Serif

Regular 400

OTF

Designed by Philipp H. Poll

OPENTYPE FEATURES NOW AFFECT BUNDLED RENDERERS.

Author	Philipp H. Poll
License	OFL
Format	otf
Weight Class	400
Styles In Collection	6
PostScript Name	LibertinusSerif-Regular
Version	Manual v1.0
Copyright	Example copyright line for the details section.

10.5. Long Title Overflow Strategy

```

#font-showcase(
  columns: 2,
  theme: (title-overflow: auto),
  fonts: (
    (
      name: "Libertinus Serif",
      display-name: "Libertinus Serif With A Deliberately Long Showcase Title For Narrow Columns",
      sample-text: "Auto overflow should stack metadata when the title gets too wide.",
    ),
    (
      path: read("Jaldi-Regular.ttf", encoding: none),
      name: "Jaldi With Another Intentionally Long Regression Title For Multi Column Cards",
      sample-text: "The fallback should preserve the original inline header for normal names.",
      description: none,
    ),
  ),
)

```

```
),  
)
```

Libertinus Serif Regular 400

Auto overflow should
stack metadata when
the title gets too wide.

Regular 400 STATIC

Jaldi With Another In- tentionally Long Regres- sion Title For Multi Col- umn Cards

Designed by Pablo Cosgaya and Nicolas Silva

The fallback should
preserve the original in-
line header for normal
names.

10.6. Waterfall, Glyphs, and Multi-Column Comparison

This multi-column example intentionally suppresses font descriptions so the non-breakable default cards stay compact and predictable.

```
#font-showcase(  
  columns: 2,  
  theme: (sample-fallback: false),  
  fonts: (  
    (  
      path: read("Jaldi-Regular.ttf", encoding: none),  
      sample-text: "Fast, readable, friendly.",  
      waterfall: (1.0em, 1.4em, 2.0em),  
      show-glyphs: true,  
      leading: 0.4em,  
      description: none,  
    ),  
    (  
      path: read("PlaywriteDKUloopetGuides-Regular.ttf", encoding: none),  
      sample-text: "Guide-aware handwriting forms.",  
      waterfall: (1.0em, 1.4em, 2.0em),  
      leading: 1.2em,  
      description: none,  
    ),  
  ),  
)
```

Jaldi

Regular 400 STATIC

Designed by Pablo Cosgaya and Nicolas Silva

1em

Fast, readable, friendly.

1.4em

Fast, readable, friendly.

2em

Fast, readable, friendly.

Numbers

0 1 2 3 4 5 6 7 8 9

Latin Uppercase

A B C D E F G H I J K L M N O P Q R S T U
V W X Y Z

Latin Lowercase

a b c d e f g h i j k l m n o p q r s t u v w x
y z

Regular 400 STATIC

Playwrite DK Uloopet Guides

Designed by Veronika Burian, José Scaglione

1em

Guide-aware handwriting forms.

1.4em

Guide-aware handwriting forms.

2em

Guide-aware handwriting forms.

10.7. Variable Font Axis Inspection

This example visualizes `variations.axes` directly. The code block shows the structure you would receive from a parsed variable font. The live result below uses the system variable font `Skia` together with the parsed axis values observed on this machine (`wght` and `wdth`). Typst 0.14.x still warns here because variable-font shaping is not supported natively yet; that warning comes from Typst itself. If you install a static instance of the same family, the warning can go away, but you still will not get true axis application. If you want to attach those requests ergonomically, use `set-variations(...)`.

```
#let variable-axis-render = it => {  
  let ft = it.font-text  
  let variations = it.at("variations", default: ({}))  
  let axes = variations.at("axes", default: ({}))  
  let coords = variations.at("coordinates", default: ({}))  
  let cells = ([*Tag*], [*Name*], [*Range*], [*Default*], [*Hidden*])  
  for axis in axes {  
    let range-label = str(axis.at("min-value", default: "?")) + " .. " + str(axis.at("max-value", default: "?"))  
    let axis-name = axis.at("name", default: none)  
    cells.push([#axis.at("tag", default: "?")])  
    cells.push([#if axis-name == none { "(unnamed)" } else { axis-name }])  
    cells.push([#range-label])  
    cells.push([#axis.at("default-value", default: "?")])  
    cells.push([#axis.at("hidden", default: false)])  
  }  
}
```

```

block(
  breakable: false,
  width: 100%,
  inset: 1.2em,
  radius: 0.6em,
  stroke: 0.5pt + rgb("d7dce5"),
  fill: rgb("fbfbfd"),
  [
    #text(size: 1.1em, weight: "bold")[#it.name]
    #v(0.35em)
    #text(size: 0.8em, fill: rgb("667085"))[
      axes=#axes.len() / normalized coordinates=#coords.map(v => str(v)).join(", ")
    ]
    #v(0.9em)
    #ft(size: 1.9em)[Variable specimen]
    #v(0.9em)
    #grid(columns: (auto, 1fr, auto, auto, auto), column-gutter: 0.8em, row-gutter:
0.35em, ..cells)
  ],
)
}

#font-showcase(
  render: variable-axis-render,
  fonts: ((
    name: "Skia",
    display-name: "Skia (System Variable Font)",
    font-type: "Variable",
    variations: (
      axes: (
        (tag: "wght", name: "Weight", min-value: 0.47999573, default-value: 1.0, max-value:
3.199997, hidden: false),
        (tag: "wdth", name: "Width", min-value: 0.61997986, default-value: 1.0, max-value:
1.300003, hidden: false),
      ),
      coordinates: (0, 0),
      has-non-default-coordinates: false,
    ),
  ),),
)

```

Skia

axes=2 / normalized coordinates=0, 0

Variable specimen

Tag	Name	Range	Default	Hidden
wght	Weight	0.47999573 .. 3.199997	1	false
wdth	Width	0.61997986 .. 1.300003	1	false

10.8. Inspector Theme

```
#import "@preview/typarium:0.1.1": font-showcase, inspector-theme, inspector-render
```

```
#font-showcase(  
  theme: inspector-theme,  
  render: inspector-render,  
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none))),  
)
```

Jaldi

glyphs=1222 / named=1222 / codepoint samples=256

Glyph metrics inspector

color: no svg: no raster: no variable: no

Glyph Probe

ID	Name	Advance	Side Bearing	BBox
0	.notdef	914	13	888 x 1244
1	NULL	0	0	-
2	non-mark- ingre- turn	427	0	-
3	space	427	0	-
4	A	1090	22	1046 x 1252
5	Aacute	1090	22	1046 x 1619
6	Abreve	1090	22	1046 x 1588
7	uni1EAE	1090	22	1046 x 1760

Codepoint Samples

0	U+30	gid=929 / zero
1	U+31	gid=930 / one
2	U+32	gid=931 / two
3	U+33	gid=932 / three
4	U+34	gid=933 / four
5	U+35	gid=934 / five
6	U+36	gid=935 / six
7	U+37	gid=936 / seven

Table Diagnostics

GPOS 16670 bytes
GSUB 31242 bytes
OS/2 96 bytes
cmap 1868 bytes
glyf 363264 bytes
head 54 bytes
hhea 36 bytes
hmtx 4886 bytes
loca 4892 bytes
maxp 32 bytes
name 1530 bytes
post 14980 bytes

10.9. Terminal Theme

```
#font-showcase(  
  fonts: ((path: read("Jaldi-Regular.ttf", encoding: none), hero-text: "SYS.INIT()")),  
  theme: terminal-theme,  
  render: terminal-render,  
)
```

```
root@typst-os:~# fetch-font-meta --name "Jaldi"
> LOADING METADATA... OK.> AUTHOR: Pablo Cosgaya and Nicolas Silva> TYPE: Static>
VERSION: Version 1.007> STYLE: 400 / normal

root@typst-os:~# render-sample --target "Jaldi"

| SYS.INIT()
| Whereas recognition of the inherent dignity

root@typst-os:~# _
```

10.10. Anatomy Theme

```
#font-showcase(
  fonts: (
    (path: read("Jaldi-Regular.ttf", encoding: none)),
    (
      path: read("PlaywriteDKUloopetGuides-Regular.ttf", encoding: none),
      theme: (visual-sample-inset-bottom: 5em),
    ),
  ),
  theme: anatomy-theme,
  render: anatomy-render,
)
```

VISUAL RATIO (Unit/Em: 2048)

Abg

ASCENT		2350u
CAP-HEIGHT		1243u
X-HEIGHT		930u
DESCENT		-1111u

Technical Specs

Format	Static
Total Glyphs	1222 units
Design UPM	2048
Weight Class	400
Width Class	5
Italic Angle	0°

Legal & Source

Copyright (c) 2011-2015, Omnibus-Type (www.omnibus-type.com omnibus.type@gmail.com) with Reserved Font Name 'Jaldi' and 'Asap'

[License URL](#) ↗

DESIGNER'S SPECIMEN

Whereas recognition of the inherent dignity

Playwrite DK Uloopet Guides

PlaywriteDKUloopetGuides-Regular

Version 1.003

VISUAL RATIO (Unit/Em: 1000)



ASCENT	1275u
CAP-HEIGHT	875u
X-HEIGHT	500u
DESCENT	-375u

Technical Specs

Format	Static
Total Glyphs	1250 units
Design UPM	1000
Weight Class	400
Width Class	5
Italic Angle	0°

Legal & Source

Copyright 2023 The Playwrite Project Authors (<https://github.com/TypeTogether/Playwrite>)

[License URL](#) ↗

DESIGNER'S SPECIMEN

Whereas recognition of the inherent dignity

11. Behavioral Notes and Caveats

- The package's public API is compact, but the resolved render payload is deliberately open-ended. Custom renderers can depend on any extra keys they add through per-font dictionaries.
- A per-font `theme` dictionary is merged on top of the showcase-level `theme`, so bundled renderers can now be tuned card by card.
- `features`, `lang`, and `script` now flow into bundled renderers through the prepared `font-text` helper.
- Metadata normalization is recursive, so nested dictionaries such as `metrics` now use the same hyphenated key style as top-level metadata.
- Weight defaults to the string `"regular"` when no parsed metadata is present, but byte-based parsing replaces it with a numeric class such as `400`. Custom renderers should tolerate either shape.
- For manual dictionary inputs without `path`, `name` controls the actual rendered font family, while `display-name` controls the card title.

- `title-overflow: auto` now uses measured header width for narrow multi-column cards inside the bundled default renderer; if you need deterministic behavior, set `title-overflow: "inline"` or `"stack"` explicitly.
- `title-overflow-badge-position` defaults to `"top"` so stacked headers place badges above the title block, but `"bottom"` is available if you prefer the earlier layout in the bundled default renderer.

12. Internal Helper Summary

12.1. `_parse-font-file(source)`

Behavior:

- accepts raw font bytes directly, or reads a string/path source as raw bytes;
- calls `font-plugin.extract_metadata(raw-bytes)` ;
- parses the returned JSON into a Typst dictionary.

Failure behavior:

- if Rust parsing fails, the WASM plugin returns `{}` ;
- the Typst helper then sees an empty dictionary and proceeds with non-WASM defaults.

12.2. `_resolve-font-meta(item, default-props, context-font)`

Responsibilities:

- classifies the input item as string, bytes, Typst 0.15+ path, or dictionary;
- detects local font sources from file extensions, path values, or a dictionary `path` key;
- computes a fallback family name from the current Typst font context;
- merges host-level defaults, resolved metadata, and explicit per-item overrides;
- normalizes metadata keys recursively from underscore to hyphen and aliases `type` to `font-type` ;
- makes sure `name` is never left as `none` .

This helper is where most of the library's real API contract lives, because it decides what the renderer eventually receives.

13. Practical Recommendations

1. Use plain strings for the fastest system-font previews.
2. Use `path` dictionaries when you want automatic metadata plus selective overrides; their `path` value can be `read(...)` bytes or Typst 0.15+ `path(...)` .
3. Use `display-name` when a manual card should show a label different from the actual system font family.
4. Reach for `theme` when the default layout is fine but the art direction should change.
5. Reach for `render` when the information architecture should change.
6. Put bundled-renderer specimen and visibility keys in `theme` when you want shared behavior without switching renderers.